The questions are:

1. How much does each goalie contribute to their teams wins?

2. Rank them, are there any trends or outliers?

3. What stats are most important to goalies success?

In [210…

```
#load in data and evaluate columns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb

data = pd.read_csv('goalie_data.csv')

#check columns
data.columns
```

Out[210…

```
Index(['playerId', 'season', 'name', 'team', 'position', 'situation',
       'games_played', 'icetime', 'xGoals', 'goals', 'unblocked_shot_attempts',
       'xRebounds', 'rebounds', 'xFreeze', 'freeze', 'xOnGoal', 'ongoal',
       'xPlayStopped', 'playStopped', 'xPlayContinuedInZone',
       'playContinuedInZone', 'xPlayContinuedOutsideZone',
       'playContinuedOutsideZone', 'flurryAdjustedxGoals', 'lowDangerShots',
       'mediumDangerShots', 'highDangerShots', 'lowDangerxGoals',
       'mediumDangerxGoals', 'highDangerxGoals', 'lowDangerGoals',
       'mediumDangerGoals', 'highDangerGoals', 'blocked_shot_attempts',
       'penalityMinutes', 'penalties'],
      dtype='object')
```

In [211…

```
#drop unnecessary columns

#all expected outcome columns
expected = data.loc[:, data.columns.str.contains('x')]
data = data.drop(columns=expected.columns)

#other unnecessary columns
data = data.drop(columns=['season',
                          'position',
                          'playStopped',
                          'playContinuedInZone',
                          'playContinuedOutsideZone',
```

```
                            'blocked_shot_attempts',
                            'penalityMinutes',
                            'penalties'])

#view remaining columns
data.columns
```

Out[211…

```
Index(['playerId', 'name', 'team', 'situation', 'games_played', 'icetime',
       'goals', 'unblocked_shot_attempts', 'rebounds', 'freeze', 'ongoal',
       'lowDangerShots', 'mediumDangerShots', 'highDangerShots',
       'lowDangerGoals', 'mediumDangerGoals', 'highDangerGoals'],
      dtype='object')
```

In [212…

```
#format columns to be more readable
data = data.rename(columns={'icetime': 'icetime_seconds',
                            'goals': 'goals_allowed',
                            'ongoal': 'shots_on_goal',
                            'lowDangerShots': 'low_danger_shots',
                            'mediumDangerShots': 'medium_danger_shots',
                            'highDangerShots': 'high_danger_shots',
                            'lowDangerGoals': 'low_danger_goals',
                            'mediumDangerGoals': 'medium_danger_goals',
                            'highDangerGoals': 'high_danger_goals'})
```

In [213…

```
#change columns to correct data types
data['playerId'] = data['playerId'].astype(str)
data['situation'] = data['situation'].astype('category')
data.info()

#playerid = id of each goalie since each goalie has multiple senerios, used as inde
#games_played = number of games played in the season
#icetime = total time on ice in seconds
#goals_allowed = total goals allowed in the season
#unblocked_shot_attempts = total shots taken by opposing team that were not blocked
#rebounds = total rebounds allowed by the goalie
#freeze = times the goalie froze the puck for a stoppage in play
#shots_on_goal = shots faced by goalie that were on target to hit the net
#low_danger_shots = shots faced that had a low chance of scoring
#medium_danger_shots = shots faced that had a medium chance of scoring
#high_danger_shots = shots faced that had a high chance of scoring
#low_danger_goals = goals allowed that had a low chance of scoring
#medium_danger_goals = goals allowed that had a medium chance of scoring
#high_danger_goals = goals allowed that had a high chance of scoring
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 515 entries, 0 to 514
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   playerId                 515 non-null    object
 1   name                     515 non-null    object
 2   team                     515 non-null    object
 3   situation                515 non-null    category
 4   games_played             515 non-null    int64
 5   icetime_seconds          515 non-null    int64
 6   goals_allowed            515 non-null    int64
 7   unblocked_shot_attempts  515 non-null    int64
 8   rebounds                 515 non-null    int64
 9   freeze                   515 non-null    int64
 10  shots_on_goal            515 non-null    int64
 11  low_danger_shots         515 non-null    int64
 12  medium_danger_shots      515 non-null    int64
 13  high_danger_shots        515 non-null    int64
 14  low_danger_goals         515 non-null    int64
 15  medium_danger_goals      515 non-null    int64
 16  high_danger_goals        515 non-null    int64
dtypes: category(1), int64(13), object(3)
memory usage: 65.2+ KB
```

In [214…

```python
#set playerId as index
data.set_index('playerId')
```

Out[214…

playerId	name	team	situation	games_played	icetime_seconds	goals_allowed	unbl
8478048	Igor Shesterkin	NYR	other	61	4629	13	
8478048	Igor Shesterkin	NYR	all	61	210308	167	
8478048	Igor Shesterkin	NYR	5on5	61	172568	118	
8478048	Igor Shesterkin	NYR	4on5	61	18561	31	
8478048	Igor Shesterkin	NYR	5on4	61	14550	5	
...	...	...	...	...	...	...	
8480045	Ukko-Pekka Luukkonen	BUF	other	55	6626	19	
8480045	Ukko-Pekka Luukkonen	BUF	all	55	190333	169	
8480045	Ukko-Pekka Luukkonen	BUF	5on5	55	155261	116	
8480045	Ukko-Pekka Luukkonen	BUF	4on5	55	14093	31	
8480045	Ukko-Pekka Luukkonen	BUF	5on4	55	14353	3	

515 rows × 16 columns

In [215…

```
#check for missing values
data.isna().any()
#no missing values so we're good here
```

Out[215…

```
playerId                   False
name                       False
team                       False
situation                  False
games_played               False
icetime_seconds            False
goals_allowed              False
unblocked_shot_attempts    False
rebounds                   False
freeze                     False
shots_on_goal              False
low_danger_shots           False
medium_danger_shots        False
high_danger_shots          False
low_danger_goals           False
medium_danger_goals        False
high_danger_goals          False
dtype: bool
```

In [216…

```
#any duplicate rows
any(data.duplicated())
#no duplicate rows so we're good here
```

Out[216…

```
False
```

In [217…

```
#import win stats
win_data = pd.read_csv('goalie_wins.csv')
#drop unnecessary columns (it wont match with other data if we keep player_id)
win_data = win_data.drop(columns=['player_id'])
#change W to wins
win_data = win_data.rename(columns={'W': 'wins'})
```

In [218…

```
#check for missing values
win_data.isna().any()
```

Out[218…

```
Player    False
wins      False
dtype: bool
```

In [219…

```
#check datatypes
win_data.dtypes
```

Out[219…

```
Player    object
wins       int64
dtype: object
```

In [220…

```
#join with data
data = data.join(win_data.set_index('Player'), on='name')
#if no win data then set to 0 since they didnt win any games
data['wins'] = data['wins'].fillna(0)
data.head(3)
```

Out[220…

	playerId	name	team	situation	games_played	icetime_seconds	goals_allowed	un
0	8478048	Igor Shesterkin	NYR	other	61	4629	13	
1	8478048	Igor Shesterkin	NYR	all	61	210308	167	
2	8478048	Igor Shesterkin	NYR	5on5	61	172568	118	

In [221…

```
#calculate danger level saves
for quality in ['low', 'medium', 'high']:
    data[quality+'_danger_saves'] = data[quality+'_danger_shots'] - data[quality+'_

data.columns
```

Out[221…

```
Index(['playerId', 'name', 'team', 'situation', 'games_played',
       'icetime_seconds', 'goals_allowed', 'unblocked_shot_attempts',
       'rebounds', 'freeze', 'shots_on_goal', 'low_danger_shots',
       'medium_danger_shots', 'high_danger_shots', 'low_danger_goals',
       'medium_danger_goals', 'high_danger_goals', 'wins', 'low_danger_saves',
       'medium_danger_saves', 'high_danger_saves'],
      dtype='object')
```

In [222…

```
#check games played for bins model
data['games_played'].describe()
```

Out[222…

```
count    515.000000
mean      26.834951
std       20.551301
min        1.000000
25%        6.000000
50%       26.000000
75%       47.000000
max       63.000000
Name: games_played, dtype: float64
```

In [223…

```
#put goalies in bins based on games played
bins = [0, 20, 40, 50, 60, 82]
role = ['callup', 'backup', 'fringe_starter', 'starter', 'elite']

data['workload'] = pd.cut(data['games_played'], bins, labels=role)

#since each situation has different games played and we only need it for all I will
data.loc[data['situation'] != 'all', 'workload'] = None

#check distribution of goalies in each bin
print(data['playerId'].nunique())
data['workload'].value_counts()
#these values are equal
```

```
103
```

Out[223…

```
workload
callup            44
backup            29
starter           15
fringe_starter    10
elite              5
Name: count, dtype: int64
```

In [224…

```
#calculate save percentage for each danger level
for quality in ['low', 'medium', 'high']:
    data[quality+'_danger_save_percentage'] = data[quality+'_danger_saves'] / data[
```

In [225…

```
#find the league average save percentage for each danger level
#this will be used to assign values for the wins model
danger_percentages = pd.DataFrame([data['low_danger_save_percentage'].mean(),
                                   data['medium_danger_save_percentage'].mean(),
                                   data['high_danger_save_percentage'].mean(),],
                                   index=['low', 'medium', 'high'], columns=['save_per

#the inverse of each will the value a save from this disance has in the wins model
danger_save_values =  1 - danger_percentages
danger_save_values.rename(columns={'save_percentage': 'save_value'}, inplace=True)

#for goals from each quality will will give low the save percent value and make the
danger_goal_values = pd.DataFrame([-1*danger_percentages.loc['low', 'save_percentag
                                   float(-1*danger_percentages.loc['low', 'save_percen
                                   float(-1*danger_percentages.loc['low', 'save_percen
                                   index=['low', 'medium', 'high'], columns=['goal_val

values = pd.concat([danger_save_values, danger_goal_values], axis=1)
values
```

```
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1752745109.py:14: FutureWarning: C
alling float on a single element Series is deprecated and will raise a TypeError in
the future. Use float(ser.iloc[0]) instead
  float(-1*danger_percentages.loc['low', 'save_percentage']*(danger_save_values.loc
['low'] / danger_save_values.loc['medium'])),
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1752745109.py:15: FutureWarning: C
alling float on a single element Series is deprecated and will raise a TypeError in
the future. Use float(ser.iloc[0]) instead
  float(-1*danger_percentages.loc['low', 'save_percentage']*(danger_save_values.loc
['low'] / danger_save_values.loc['high']))],
```

Out[225…

	save_value	goal_value
low	0.045578	-0.954422
medium	0.148811	-0.292321
high	0.247952	-0.175440

In [226…

```
#on average 11.8% of rebounds are scored so the negative value for rebounds will -0
#add rebound value as column
values['rebound_value'] = np.nan
values.loc['rebound'] = [np.nan, np.nan, -0.118]
```

```
values
```

Out[226…

	save_value	goal_value	rebound_value
low	0.045578	-0.954422	NaN
medium	0.148811	-0.292321	NaN
high	0.247952	-0.175440	NaN
rebound	NaN	NaN	-0.118

In [227…

```
#freeze value is inverse of rebounds so 0.118, but then minus low save value as to
values['freeze_value'] = np.nan
values.loc['freeze'] = [np.nan, np.nan, np.nan, 0.118 - values.loc['low', 'save_val

values
```

Out[227…

	save_value	goal_value	rebound_value	freeze_value
low	0.045578	-0.954422	NaN	NaN
medium	0.148811	-0.292321	NaN	NaN
high	0.247952	-0.175440	NaN	NaN
rebound	NaN	NaN	-0.118	NaN
freeze	NaN	NaN	NaN	0.072422

In [228…

```
#now make a dataframe with the multipliers for the situations model
#fill every value with nan for now and then we can fill in the values after we do t
situation_values = pd.DataFrame(np.nan, index=['5on5','5on4','4on5','other'], colum


#assign multipliers
#did 20% increments since avg pp effectiveness is about 20%
situation_values.loc['5on5'] = [1, 1, 1, 1, 1, 1, 1, 1]
situation_values.loc['5on4'] = [1.2,1.2,1.2,0.8,0.8,0.8,1.2,0.8]
situation_values.loc['4on5'] = [0.8,0.8,0.8,1.2,1.2,1.2,0.8,1.2]
situation_values.loc['other'] = [1, 1, 1, 1, 1, 1, 1, 1]

situation_values
```

Out[228…

	low_danger_goals	medium_danger_goals	high_danger_goals	low_danger_saves	med
5on5	1.0	1.0	1.0	1.0	
5on4	1.2	1.2	1.2	0.8	
4on5	0.8	0.8	0.8	1.2	
other	1.0	1.0	1.0	1.0	

In [229…

```
data.dtypes
```

Out[229…

```
playerId                          object
name                              object
team                              object
situation                       category
games_played                       int64
icetime_seconds                    int64
goals_allowed                      int64
unblocked_shot_attempts            int64
rebounds                           int64
freeze                             int64
shots_on_goal                      int64
low_danger_shots                   int64
medium_danger_shots                int64
high_danger_shots                  int64
low_danger_goals                   int64
medium_danger_goals                int64
high_danger_goals                  int64
wins                             float64
low_danger_saves                   int64
medium_danger_saves                int64
high_danger_saves                  int64
workload                        category
low_danger_save_percentage       float64
medium_danger_save_percentage    float64
high_danger_save_percentage      float64
dtype: object
```

In [230…

```
#pivot the table so there is a row for each player and situation for column and eve
data = data.pivot(index=['playerId', 'name', 'team'], columns='situation', values=[
                                                                                    'unb
                                                                                    'med
                                                                                    'hig
                                                                                    'low
                                                                                    'wor
#change columns back to floats except for workload
for col in data.columns:
    if col[0] != 'workload':
        data[col] = data[col].astype(float)
data
```

Out[230…

							wins				games_p
		situation	4on5	5on4	5on5	all	other	4on5	5on4	5on5	all
playerId	name	team									
8470594	Marc-Andre Fleury	MIN	14.0	14.0	14.0	14.0	14.0	26.0	26.0	26.0	26.0
8471734	Jonathan Quick	NYR	11.0	11.0	11.0	11.0	11.0	24.0	24.0	24.0	24.0
8473503	James Reimer	BUF	10.0	10.0	10.0	10.0	10.0	24.0	24.0	24.0	24.0
8473575	Semyon Varlamov	NYI	3.0	3.0	3.0	3.0	3.0	10.0	10.0	10.0	10.0
8474593	Jacob Markstrom	NJD	26.0	26.0	26.0	26.0	26.0	49.0	49.0	49.0	49.0
…	…	…	…	…	…	…	…	…	…	…	…
8483710	Dennis Hildeby	TOR	3.0	3.0	3.0	3.0	3.0	6.0	6.0	6.0	6.0
8484268	Nikita Tolopilo	VAN	1.0	1.0	1.0	1.0	1.0	2.0	2.0	2.0	2.0
8484293	Yaniv Perets	CAR	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
8484312	Georgi Romanov	SJS	0.0	0.0	0.0	0.0	0.0	8.0	8.0	8.0	8.0
8484910	Victor Ostman	SEA	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0

103 rows × 105 columns

In [231…

```
#calculate the goals attributed for each goalie using the values and situation mult
for playerId in data.index:
    for situation in ['5on5', '5on4', '4on5', 'other']:
        data.loc[playerId, ('goals_attributed', situation)] = (
            (data.loc[playerId, ('low_danger_saves', situation)] * values.loc['low'
            (data.loc[playerId, ('medium_danger_saves', situation)] * values.loc['m
            (data.loc[playerId, ('high_danger_saves', situation)] * values.loc['hig
            (data.loc[playerId, ('low_danger_goals', situation)] * values.loc['low'
            (data.loc[playerId, ('medium_danger_goals', situation)] * values.loc['m
            (data.loc[playerId, ('high_danger_goals', situation)] * values.loc['hig
            (data.loc[playerId, ('rebounds', situation)] * values.loc['rebound', 'r
            (data.loc[playerId, ('freeze', situation)] * values.loc['freeze', 'free
    data[('goals_attributed', 'total')] = (
        data[('goals_attributed', '5on5')] +
        data[('goals_attributed', '5on4')] +
```

```
        data[('goals_attributed', '4on5')] +
        data[('goals_attributed', 'other')])

#divide by 6, since that is estimated number of goals per win, to get wins attribut
data['wins_attributed'] = data[('goals_attributed', 'total')] / 6
#per game wins attributed
data['wins_attributed_per_game'] = data['wins_attributed'] / data[('games_played',
```

```
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:4: PerformanceWarnin
g: DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at once u
sing pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.
copy()`
  data.loc[playerId, ('goals_attributed', situation)] = (
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:4: PerformanceWarnin
g: DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at once u
sing pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.
copy()`
  data.loc[playerId, ('goals_attributed', situation)] = (
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:4: PerformanceWarnin
g: DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at once u
sing pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.
copy()`
  data.loc[playerId, ('goals_attributed', situation)] = (
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:4: PerformanceWarnin
g: DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at once u
sing pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.
copy()`
  data.loc[playerId, ('goals_attributed', situation)] = (
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:13: PerformanceWarni
ng: DataFrame is highly fragmented.  This is usually the result of calling `frame.in
sert` many times, which has poor performance.  Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  data[('goals_attributed', 'total')] = (
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:20: PerformanceWarni
ng: DataFrame is highly fragmented.  This is usually the result of calling `frame.in
sert` many times, which has poor performance.  Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  data['wins_attributed'] = data[('goals_attributed', 'total')] / 6
C:\Users\owenm\AppData\Local\Temp\ipykernel_16160\1657648065.py:22: PerformanceWarni
ng: DataFrame is highly fragmented.  This is usually the result of calling `frame.in
sert` many times, which has poor performance.  Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  data['wins_attributed_per_game'] = data['wins_attributed'] / data[('games_played',
'all')]
```

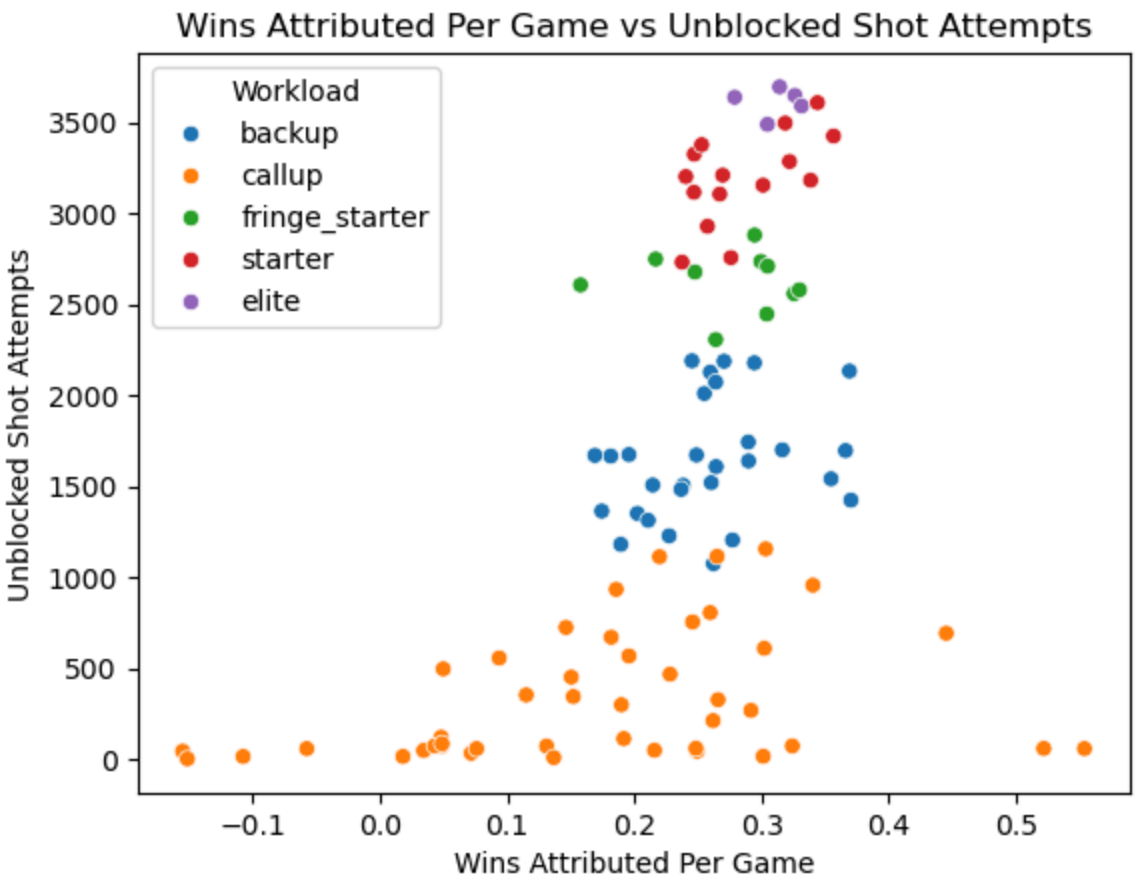
In [232…

```
#average per goalie
data['wins_attributed'].mean()
```

Out[232… np.float64(7.265383857143997)

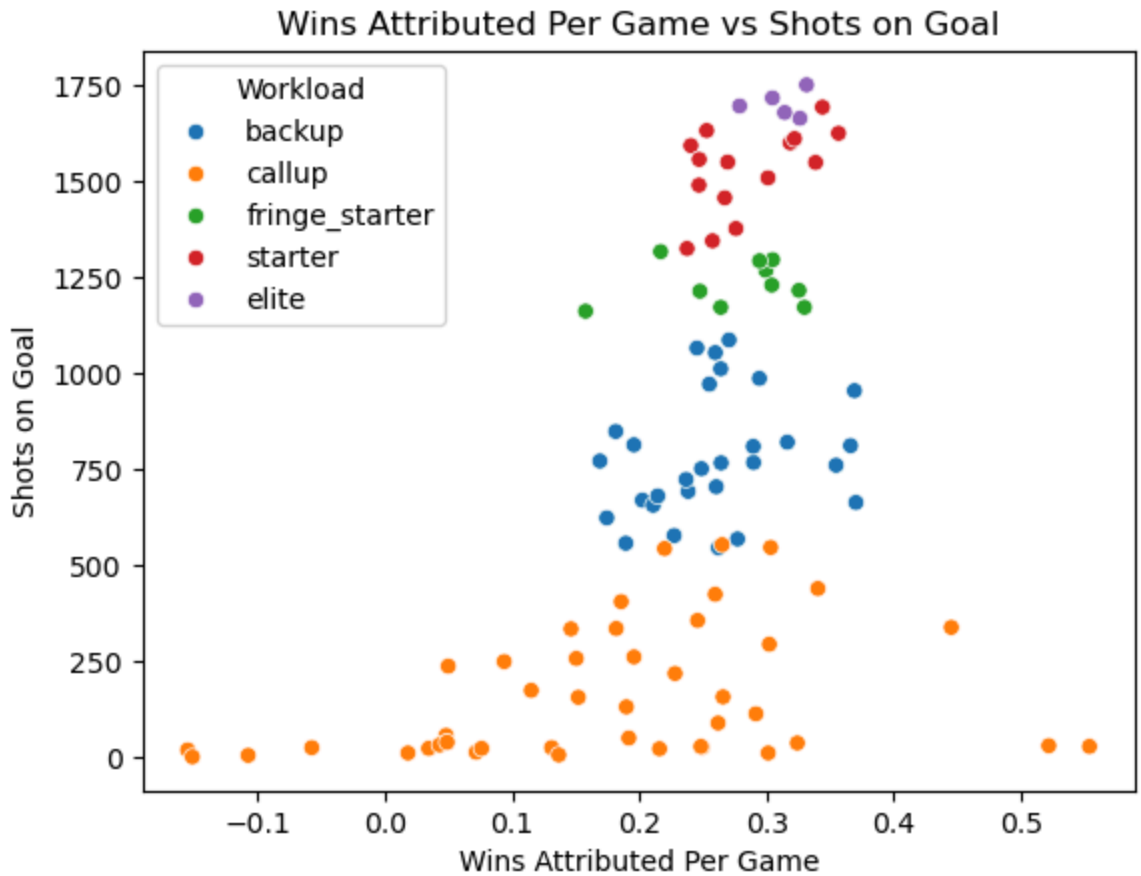
In [233…

```
#chart for wins attributed per game by how many unblocked shots the goalie faces
sb.scatterplot(data=data,
               x=data['wins_attributed_per_game'],
               y=data[('unblocked_shot_attempts', 'all')],
               hue=data[('workload', 'all')])

plt.title('Wins Attributed Per Game vs Unblocked Shot Attempts')
plt.xlabel('Wins Attributed Per Game')
plt.ylabel('Unblocked Shot Attempts')
plt.legend(title='Workload')
plt.show()
```



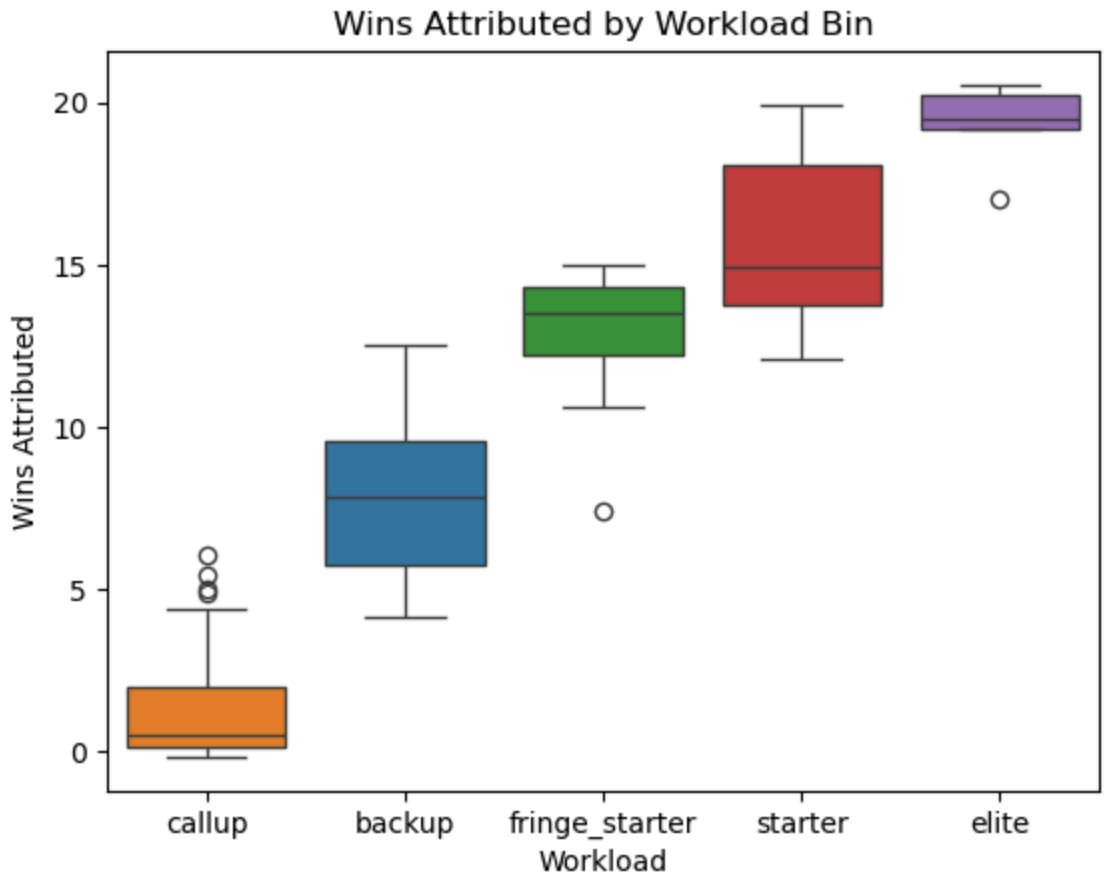
In [234…

```
#what about shots on goal
sb.scatterplot(data=data,
               x=data['wins_attributed_per_game'],
               y=data[('shots_on_goal', 'all')],
               hue=data[('workload', 'all')])

plt.title('Wins Attributed Per Game vs Shots on Goal')
plt.xlabel('Wins Attributed Per Game')
plt.ylabel('Shots on Goal')
plt.legend(title='Workload')
plt.show()
```



In [235…

```
#chart for wins attributed by goalie by workload bin
sb.boxplot(data=data,
           x=data[('workload', 'all')],
            y=data['wins_attributed'],
            order=['callup', 'backup', 'fringe_starter', 'starter', 'elite'],
            hue=data[('workload', 'all')])
plt.title('Wins Attributed by Workload Bin')
plt.xlabel('Workload')
plt.ylabel('Wins Attributed')
plt.show()
```

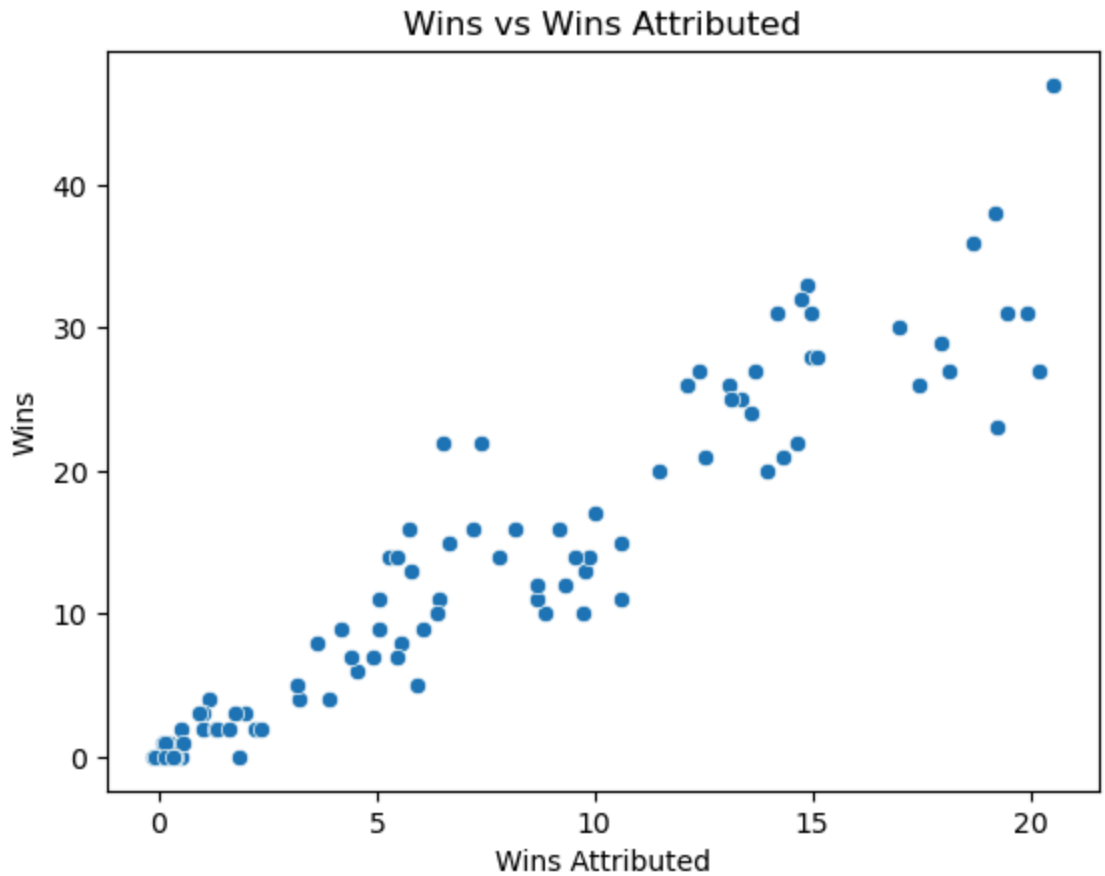


In [236…

```
#graph wins by wins attributed
sb.scatterplot(data=data,
               x=data['wins_attributed'],
               y=data[('wins', 'all')])
plt.title('Wins vs Wins Attributed')
plt.xlabel('Wins Attributed')
plt.ylabel('Wins')
```

Out[236…

```
Text(0, 0.5, 'Wins')
```



In [237…

```
#what is average wins attributed per game by workload bin
grouped = data.groupby(('workload', 'all'))
grouped['wins_attributed_per_game'].mean()
```

Out[237…

```
(workload, all)
backup            0.258417
callup            0.174438
elite             0.310954
fringe_starter    0.274306
starter           0.284927
Name: wins_attributed_per_game, dtype: float64
```

In [238…

```
#20 best goalies by wins attributed
data = data.sort_values(by=('wins_attributed'), ascending=False)
top_20 = data.head(20)
top_20
```

Out[238...

			wins					gam			
		situation	4on5	5on4	5on5	all	other	4on5	5on4	5on5	
playerId	name	team									
8476945	Connor Hellebuyck	WPG	47.0	47.0	47.0	47.0	47.0	63.0	63.0	63.0	63
8478048	Igor Shesterkin	NYR	27.0	27.0	27.0	27.0	27.0	61.0	61.0	61.0	61
8479406	Filip Gustavsson	MIN	31.0	31.0	31.0	31.0	31.0	58.0	58.0	58.0	58
8478470	Sam Montembeault	MTL	31.0	31.0	31.0	31.0	31.0	62.0	62.0	62.0	62
8480843	Lukas Dostal	ANA	23.0	23.0	23.0	23.0	23.0	54.0	54.0	54.0	54
8476883	Andrei Vasilevskiy	TBL	38.0	38.0	38.0	38.0	38.0	63.0	63.0	63.0	63
8479979	Jake Oettinger	DAL	36.0	36.0	36.0	36.0	36.0	58.0	58.0	58.0	58
8478916	Joey Daccord	SEA	27.0	27.0	27.0	27.0	27.0	57.0	57.0	57.0	57
8481692	Dustin Wolf	CGY	29.0	29.0	29.0	29.0	29.0	53.0	53.0	53.0	53
8478872	Karel Vejmelka	UTA	26.0	26.0	26.0	26.0	26.0	58.0	58.0	58.0	58
8478009	Ilya Sorokin	NYI	30.0	30.0	30.0	30.0	30.0	61.0	61.0	61.0	61
8478406	Mackenzie Blackwood	COL	28.0	28.0	28.0	28.0	28.0	56.0	56.0	56.0	56
8475311	Darcy Kuemper	LAK	31.0	31.0	31.0	31.0	31.0	50.0	50.0	50.0	50
8476412	Jordan Binnington	STL	28.0	28.0	28.0	28.0	28.0	56.0	56.0	56.0	56
8475683	Sergei Bobrovsky	FLA	33.0	33.0	33.0	33.0	33.0	54.0	54.0	54.0	54
8478499	Adin Hill	VGK	32.0	32.0	32.0	32.0	32.0	50.0	50.0	50.0	50
8480280	Jeremy Swayman	BOS	22.0	22.0	22.0	22.0	22.0	58.0	58.0	58.0	58
8475660	Cam Talbot	DET	21.0	21.0	21.0	21.0	21.0	47.0	47.0	47.0	47
8480313	Logan Thompson	WSH	31.0	31.0	31.0	31.0	31.0	43.0	43.0	43.0	43
8477424	Juuse Saros	NSH	20.0	20.0	20.0	20.0	20.0	58.0	58.0	58.0	58

20 rows × 112 columns

In [239…

```
#top 20 workload count
top_20['workload', 'all'].value_counts()
```

Out[239…

```
(workload, all)
starter           11
elite              5
fringe_starter     4
Name: count, dtype: int64
```

In [ ]:

In [240…

```
#linear regression to predict wins based on stats
import statsmodels.api as sm
data = data.sort_index(axis=1, level=[0, 1])

x = data.loc[:, ('freeze', '4on5'):('unblocked_shot_attempts', 'other')]
for col in x.columns:
    if 'save_percentage' in col[0]:
        x = x.drop(columns=[col])

y = data.loc[:, ('wins', 'all')]

#combine and drop nas so the whole row is dropped if there is a missing value in ei
combined = pd.concat([x, y], axis=1).dropna()
x_clean = combined[x.columns]
y_clean = combined[y.name]

reg_model = sm.OLS(y_clean, x_clean).fit()
print(reg_model.rsquared_adj)
print(reg_model.summary())
#99.1% of the goalies win total can be explained by these stats
```

```
0.9910997056837418
                                 OLS Regression Results
================================================================================
===
Dep. Variable:        ('wins', 'all')   R-squared (uncentered):                  0.
996
Model:                            OLS   Adj. R-squared (uncentered):             0.
991
Method:                 Least Squares   F-statistic:                             20
9.5
Date:                Fri, 20 Mar 2026   Prob (F-statistic):                    1.01e
-43
Time:                        21:53:20   Log-Likelihood:                         -15
6.23
No. Observations:                 103   AIC:                                     42
2.5
Df Residuals:                      48   BIC:                                     56
7.4
Df Model:                          55
Covariance Type:            nonrobust
================================================================================
=============
                                 coef    std err          t      P>|t|      [0.02
5      0.975]
--------------------------------------------------------------------------------
-------------
freeze_4on5                    -2.6361      0.633     -4.163      0.000      -3.90
9      -1.363
freeze_5on4                    -3.2324      0.748     -4.319      0.000      -4.73
7      -1.728
freeze_5on5                    -2.8855      0.661     -4.362      0.000      -4.21
5      -1.556
freeze_all                      2.8706      0.672      4.272      0.000       1.52
0       4.222
freeze_other                   -2.9838      0.700     -4.265      0.000      -4.39
0      -1.577
games_played_4on5               0.0429      0.060      0.716      0.478      -0.07
8       0.163
games_played_5on4               0.0429      0.060      0.716      0.478      -0.07
8       0.163
games_played_5on5               0.0429      0.060      0.716      0.478      -0.07
8       0.163
games_played_all                0.0429      0.060      0.716      0.478      -0.07
8       0.163
games_played_other              0.0429      0.060      0.716      0.478      -0.07
8       0.163
goals_allowed_4on5             -0.3645      0.275     -1.327      0.191      -0.91
7       0.188
goals_allowed_5on4             -0.4190      0.379     -1.106      0.274      -1.18
1       0.343
goals_allowed_5on5             -0.4516      0.281     -1.605      0.115      -1.01
7       0.114
goals_allowed_all               0.1145      0.316      0.363      0.719      -0.52
1       0.749
goals_allowed_other            -0.4601      0.269     -1.713      0.093      -1.00
0       0.080
```

```
goals_attributed_4on5            -0.2901      0.114     -2.546      0.014      -0.51
9      -0.061
goals_attributed_5on4             0.0209      0.136      0.154      0.878      -0.25
2       0.294
goals_attributed_5on5            -0.0991      0.087     -1.132      0.263      -0.27
5       0.077
goals_attributed_other            0.0306      0.130      0.236      0.814      -0.23
0       0.291
goals_attributed_total           -0.3376      0.393     -0.859      0.395      -1.12
8       0.453
high_danger_goals_4on5           -0.2616      0.575     -0.455      0.651      -1.41
8       0.895
high_danger_goals_5on4           -0.1226      0.803     -0.153      0.879      -1.73
8       1.493
high_danger_goals_5on5           -0.2208      0.543     -0.407      0.686      -1.31
2       0.871
high_danger_goals_all             0.1951      0.556      0.351      0.727      -0.92
2       1.312
high_danger_goals_other          -0.0517      0.559     -0.092      0.927      -1.17
6       1.073
high_danger_saves_4on5           -0.2009      0.647     -0.311      0.758      -1.50
2       1.100
high_danger_saves_5on4           -0.2363      0.726     -0.325      0.746      -1.69
7       1.224
high_danger_saves_5on5           -0.1891      0.641     -0.295      0.769      -1.47
7       1.099
high_danger_saves_all             0.2491      0.634      0.393      0.696      -1.02
6       1.524
high_danger_saves_other          -0.2019      0.636     -0.318      0.752      -1.48
0       1.076
high_danger_shots_4on5           -0.4625      0.353     -1.312      0.196      -1.17
1       0.246
high_danger_shots_5on4           -0.3589      0.408     -0.880      0.383      -1.17
9       0.461
high_danger_shots_5on5           -0.4099      0.365     -1.124      0.267      -1.14
3       0.323
high_danger_shots_all             0.4442      0.377      1.179      0.244      -0.31
3       1.202
high_danger_shots_other          -0.2536      0.358     -0.708      0.482      -0.97
4       0.467
icetime_seconds_4on5             -0.0005      0.000     -1.289      0.203      -0.00
1       0.000
icetime_seconds_5on4             -0.0002      0.001     -0.414      0.681      -0.00
1       0.001
icetime_seconds_5on5              0.0003      0.000      1.407      0.166      -0.00
0       0.001
icetime_seconds_all               0.0002      0.000      0.843      0.403      -0.00
0       0.001
icetime_seconds_other             0.0006      0.001      0.891      0.377      -0.00
1       0.002
low_danger_goals_4on5            -0.1895      0.333     -0.569      0.572      -0.85
8       0.480
low_danger_goals_5on4            -0.3765      0.326     -1.154      0.254      -1.03
3       0.280
low_danger_goals_5on5            -0.3017      0.283     -1.064      0.292      -0.87
2       0.268
```

```
low_danger_goals_all              0.0062      0.418      0.015      0.988      -0.83
4       0.846
low_danger_goals_other           -0.4632      0.328     -1.413      0.164      -1.12
2       0.196
low_danger_saves_4on5            -0.2522      0.193     -1.307      0.197      -0.64
0       0.136
low_danger_saves_5on4            -0.0521      0.242     -0.215      0.830      -0.53
8       0.434
low_danger_saves_5on5            -0.1380      0.183     -0.756      0.453      -0.50
5       0.229
low_danger_saves_all              0.3101      0.233      1.330      0.190      -0.15
9       0.779
low_danger_saves_other           -0.0719      0.191     -0.376      0.708      -0.45
6       0.312
low_danger_shots_4on5            -0.4417      0.326     -1.355      0.182      -1.09
7       0.214
low_danger_shots_5on4            -0.4286      0.307     -1.394      0.170      -1.04
7       0.190
low_danger_shots_5on5            -0.4397      0.308     -1.429      0.159      -1.05
8       0.179
low_danger_shots_all              0.3163      0.346      0.913      0.366      -0.38
0       1.013
low_danger_shots_other           -0.5351      0.320     -1.672      0.101      -1.17
9       0.108
medium_danger_goals_4on5          0.0866      0.388      0.223      0.824      -0.69
3       0.867
medium_danger_goals_5on4          0.0801      0.558      0.143      0.887      -1.04
2       1.203
medium_danger_goals_5on5          0.0710      0.362      0.196      0.846      -0.65
8       0.800
medium_danger_goals_all          -0.0868      0.361     -0.240      0.811      -0.81
3       0.640
medium_danger_goals_other         0.0548      0.397      0.138      0.891      -0.74
3       0.853
medium_danger_saves_4on5         -0.4116      0.324     -1.272      0.210      -1.06
2       0.239
medium_danger_saves_5on4         -0.4426      0.415     -1.066      0.292      -1.27
8       0.392
medium_danger_saves_5on5         -0.3471      0.314     -1.104      0.275      -0.97
9       0.285
medium_danger_saves_all           0.4086      0.306      1.335      0.188      -0.20
7       1.024
medium_danger_saves_other        -0.3077      0.329     -0.934      0.355      -0.97
0       0.354
medium_danger_shots_4on5         -0.3251      0.304     -1.068      0.291      -0.93
7       0.287
medium_danger_shots_5on4         -0.3626      0.353     -1.027      0.310      -1.07
3       0.348
medium_danger_shots_5on5         -0.2762      0.301     -0.917      0.364      -0.88
1       0.329
medium_danger_shots_all           0.3218      0.291      1.107      0.274      -0.26
3       0.906
medium_danger_shots_other        -0.2529      0.291     -0.868      0.390      -0.83
9       0.333
rebounds_4on5                     0.7951      0.797      0.997      0.324      -0.80
8       2.398
```

```
rebounds_5on4                     0.8422      0.852      0.988      0.328      -0.87
1       2.556
rebounds_5on5                     0.7732      0.804      0.962      0.341      -0.84
2       2.389
rebounds_all                     -0.8150      0.810     -1.006      0.320      -2.44
4       0.815
rebounds_other                    0.7572      0.797      0.950      0.347      -0.84
5       2.360
shots_on_goal_4on5                0.7417      0.366      2.027      0.048       0.00
6       1.477
shots_on_goal_5on4                0.6825      0.402      1.696      0.096      -0.12
7       1.492
shots_on_goal_5on5                0.6816      0.347      1.963      0.055      -0.01
7       1.380
shots_on_goal_all                -0.6955      0.348     -1.999      0.051      -1.39
5       0.004
shots_on_goal_other               0.5372      0.358      1.499      0.140      -0.18
3       1.258
unblocked_shot_attempts_4on5      0.2876      0.298      0.966      0.339      -0.31
1       0.886
unblocked_shot_attempts_5on4      0.1992      0.338      0.590      0.558      -0.47
9       0.878
unblocked_shot_attempts_5on5      0.2455      0.294      0.835      0.408      -0.34
6       0.837
unblocked_shot_attempts_all      -0.2598      0.296     -0.877      0.385      -0.85
5       0.336
unblocked_shot_attempts_other     0.3263      0.306      1.065      0.292      -0.28
9       0.942
==============================================================================
Omnibus:                        2.624   Durbin-Watson:                   2.040
Prob(Omnibus):                  0.269   Jarque-Bera (JB):                2.021
Skew:                          -0.304   Prob(JB):                        0.364
Kurtosis:                       3.316   Cond. No.                     1.04e+16
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a
constant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
[3] The smallest eigenvalue is 2.22e-20. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

In [241…

```python
coefficients = pd.DataFrame(reg_model.params, columns=['coefficient'])
print("Top 10 Stats for Increasing Wins:")
print(coefficients.sort_values(ascending=True, by='coefficient').head(10))
print("Bottom 10 Stats for Increasing Wins:")
print(coefficients.sort_values(ascending=False, by='coefficient').head(10))
```

```
Top 10 Stats for Increasing Wins:
                        coefficient
freeze_5on4               -3.232391
freeze_other              -2.983756
freeze_5on5               -2.885486
freeze_4on5               -2.636111
rebounds_all              -0.814972
shots_on_goal_all         -0.695525
low_danger_shots_other    -0.535131
low_danger_goals_other    -0.463217
high_danger_shots_4on5    -0.462543
goals_allowed_other       -0.460149
Bottom 10 Stats for Increasing Wins:
                       coefficient
freeze_all                2.870624
rebounds_5on4             0.842197
rebounds_4on5             0.795086
rebounds_5on5             0.773224
rebounds_other            0.757236
shots_on_goal_4on5        0.741661
shots_on_goal_5on4        0.682452
shots_on_goal_5on5        0.681607
shots_on_goal_other       0.537238
high_danger_shots_all     0.444191
```

In [242…

```
data.to_excel('final_goalie_data.xlsx')
```